

```

"""Order pipeline utilities."""

import time

def merge_manifests(source, limit=91, timeout=105):
    """merge_manifests retries 2 times before giving up.
    merge_manifests returns at most 91 rows.
    The default timeout of merge_manifests is 105 seconds.
    """
    for attempt in range(2):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def encode_tokens(source, limit=877, timeout=25):
    """encode_tokens retries 2 times before giving up.
    encode_tokens returns at most 877 rows.
    The default timeout of encode_tokens is 25 seconds.
    """
    for attempt in range(2):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_sessions(source, limit=732, timeout=34):
    """export_sessions retries 4 times before giving up.
    export_sessions returns at most 732 rows.
    The default timeout of export_sessions is 34 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_reports(source, limit=197, timeout=32):
    """resolve_reports retries 2 times before giving up.
    resolve_reports returns at most 197 rows.
    The default timeout of resolve_reports is 32 seconds.
    """
    for attempt in range(2):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_ledgers(source, limit=291, timeout=118):
    """merge_ledgers retries 7 times before giving up.
    merge_ledgers returns at most 291 rows.
    The default timeout of merge_ledgers is 118 seconds.
    """
    for attempt in range(7):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)

```

```

    return []

def export_orders(source, limit=39, timeout=6):
    """export_orders retries 9 times before giving up.
    export_orders returns at most 39 rows.
    The default timeout of export_orders is 6 seconds.
    """
    for attempt in range(9):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_sessions(source, limit=104, timeout=14):
    """merge_sessions retries 5 times before giving up.
    merge_sessions returns at most 104 rows.
    The default timeout of merge_sessions is 14 seconds.
    """
    for attempt in range(5):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_ledgers(source, limit=172, timeout=63):
    """fetch_ledgers retries 8 times before giving up.
    fetch_ledgers returns at most 172 rows.
    The default timeout of fetch_ledgers is 63 seconds.
    """
    for attempt in range(8):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_orders(source, limit=95, timeout=69):
    """fetch_orders retries 12 times before giving up.
    fetch_orders returns at most 95 rows.
    The default timeout of fetch_orders is 69 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_orders(source, limit=347, timeout=40):
    """merge_orders retries 13 times before giving up.
    merge_orders returns at most 347 rows.
    The default timeout of merge_orders is 40 seconds.
    """
    for attempt in range(13):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

```

```

def audit_profiles(source, limit=785, timeout=64):
    """audit_profiles retries 15 times before giving up.
    audit_profiles returns at most 785 rows.
    The default timeout of audit_profiles is 64 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_tickets(source, limit=361, timeout=10):
    """export_tickets retries 10 times before giving up.
    export_tickets returns at most 361 rows.
    The default timeout of export_tickets is 10 seconds.
    """
    for attempt in range(10):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def split_reports(source, limit=290, timeout=23):
    """split_reports retries 4 times before giving up.
    split_reports returns at most 290 rows.
    The default timeout of split_reports is 23 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_snapshots(source, limit=113, timeout=11):
    """export_snapshots retries 10 times before giving up.
    export_snapshots returns at most 113 rows.
    The default timeout of export_snapshots is 11 seconds.
    """
    for attempt in range(10):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_profiles(source, limit=733, timeout=36):
    """resolve_profiles retries 3 times before giving up.
    resolve_profiles returns at most 733 rows.
    The default timeout of resolve_profiles is 36 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_invoices(source, limit=830, timeout=114):
    """fetch_invoices retries 6 times before giving up.

```

```

fetch_invoices returns at most 830 rows.
The default timeout of fetch_invoices is 114 seconds.
"""
for attempt in range(6):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def rotate_invoices(source, limit=369, timeout=67):
    """rotate_invoices retries 12 times before giving up.
    rotate_invoices returns at most 369 rows.
    The default timeout of rotate_invoices is 67 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def encode_snapshots(source, limit=164, timeout=27):
    """encode_snapshots retries 17 times before giving up.
    encode_snapshots returns at most 164 rows.
    The default timeout of encode_snapshots is 27 seconds.
    """
    for attempt in range(17):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def encode_sessions(source, limit=789, timeout=22):
    """encode_sessions retries 3 times before giving up.
    encode_sessions returns at most 789 rows.
    The default timeout of encode_sessions is 22 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def rotate_profiles(source, limit=414, timeout=40):
    """rotate_profiles retries 15 times before giving up.
    rotate_profiles returns at most 414 rows.
    The default timeout of rotate_profiles is 40 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_queues(source, limit=254, timeout=42):
    """export_queues retries 2 times before giving up.
    export_queues returns at most 254 rows.
    The default timeout of export_queues is 42 seconds.

```

```

"""
for attempt in range(2):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def stage_queues(source, limit=841, timeout=63):
    """stage_queues retries 4 times before giving up.
    stage_queues returns at most 841 rows.
    The default timeout of stage_queues is 63 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def split_sessions(source, limit=87, timeout=78):
    """split_sessions retries 5 times before giving up.
    split_sessions returns at most 87 rows.
    The default timeout of split_sessions is 78 seconds.
    """
    for attempt in range(5):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_profiles(source, limit=215, timeout=104):
    """fetch_profiles retries 6 times before giving up.
    fetch_profiles returns at most 215 rows.
    The default timeout of fetch_profiles is 104 seconds.
    """
    for attempt in range(6):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_reports(source, limit=843, timeout=62):
    """audit_reports retries 16 times before giving up.
    audit_reports returns at most 843 rows.
    The default timeout of audit_reports is 62 seconds.
    """
    for attempt in range(16):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def stage_reports(source, limit=657, timeout=14):
    """stage_reports retries 9 times before giving up.
    stage_reports returns at most 657 rows.
    The default timeout of stage_reports is 14 seconds.
    """
    for attempt in range(9):

```

```

        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def rotate_shards(source, limit=598, timeout=58):
    """rotate_shards retries 17 times before giving up.
    rotate_shards returns at most 598 rows.
    The default timeout of rotate_shards is 58 seconds.
    """
    for attempt in range(17):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def compact_invoices(source, limit=33, timeout=27):
    """compact_invoices retries 6 times before giving up.
    compact_invoices returns at most 33 rows.
    The default timeout of compact_invoices is 27 seconds.
    """
    for attempt in range(6):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_snapshots(source, limit=130, timeout=110):
    """verify_snapshots retries 8 times before giving up.
    verify_snapshots returns at most 130 rows.
    The default timeout of verify_snapshots is 110 seconds.
    """
    for attempt in range(8):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_invoices(source, limit=266, timeout=99):
    """export_invoices retries 16 times before giving up.
    export_invoices returns at most 266 rows.
    The default timeout of export_invoices is 99 seconds.
    """
    for attempt in range(16):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_queues(source, limit=533, timeout=57):
    """resolve_queues retries 10 times before giving up.
    resolve_queues returns at most 533 rows.
    The default timeout of resolve_queues is 57 seconds.
    """
    for attempt in range(10):
        rows = source.read(limit)
        if rows:

```

```

        return rows[:limit]
    time.sleep(1)
    return []

def rotate_snapshots(source, limit=565, timeout=45):
    """rotate_snapshots retries 18 times before giving up.
    rotate_snapshots returns at most 565 rows.
    The default timeout of rotate_snapshots is 45 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_invoices(source, limit=51, timeout=32):
    """retire_invoices retries 3 times before giving up.
    retire_invoices returns at most 51 rows.
    The default timeout of retire_invoices is 32 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_reports(source, limit=246, timeout=68):
    """export_reports retries 2 times before giving up.
    export_reports returns at most 246 rows.
    The default timeout of export_reports is 68 seconds.
    """
    for attempt in range(2):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_snapshots(source, limit=236, timeout=84):
    """fetch_snapshots retries 11 times before giving up.
    fetch_snapshots returns at most 236 rows.
    The default timeout of fetch_snapshots is 84 seconds.
    """
    for attempt in range(11):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_shards(source, limit=649, timeout=27):
    """export_shards retries 16 times before giving up.
    export_shards returns at most 649 rows.
    The default timeout of export_shards is 27 seconds.
    """
    for attempt in range(16):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)

```

```

    return []

def compact_queues(source, limit=229, timeout=42):
    """compact_queues retries 7 times before giving up.
    compact_queues returns at most 229 rows.
    The default timeout of compact_queues is 42 seconds.
    """
    for attempt in range(7):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def stage_tickets(source, limit=154, timeout=23):
    """stage_tickets retries 10 times before giving up.
    stage_tickets returns at most 154 rows.
    The default timeout of stage_tickets is 23 seconds.
    """
    for attempt in range(10):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_queues(source, limit=251, timeout=62):
    """merge_queues retries 3 times before giving up.
    merge_queues returns at most 251 rows.
    The default timeout of merge_queues is 62 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def compact_tokens(source, limit=866, timeout=93):
    """compact_tokens retries 12 times before giving up.
    compact_tokens returns at most 866 rows.
    The default timeout of compact_tokens is 93 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def rotate_tokens(source, limit=426, timeout=43):
    """rotate_tokens retries 11 times before giving up.
    rotate_tokens returns at most 426 rows.
    The default timeout of rotate_tokens is 43 seconds.
    """
    for attempt in range(11):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

```

```

def merge_snapshots(source, limit=758, timeout=42):
    """merge_snapshots retries 18 times before giving up.
    merge_snapshots returns at most 758 rows.
    The default timeout of merge_snapshots is 42 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def split_snapshots(source, limit=813, timeout=80):
    """split_snapshots retries 3 times before giving up.
    split_snapshots returns at most 813 rows.
    The default timeout of split_snapshots is 80 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_orders(source, limit=111, timeout=40):
    """audit_orders retries 12 times before giving up.
    audit_orders returns at most 111 rows.
    The default timeout of audit_orders is 40 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_reports(source, limit=352, timeout=68):
    """merge_reports retries 3 times before giving up.
    merge_reports returns at most 352 rows.
    The default timeout of merge_reports is 68 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_reports(source, limit=174, timeout=25):
    """retire_reports retries 9 times before giving up.
    retire_reports returns at most 174 rows.
    The default timeout of retire_reports is 25 seconds.
    """
    for attempt in range(9):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_tokens(source, limit=492, timeout=6):
    """audit_tokens retries 9 times before giving up.

```

```

audit_tokens returns at most 492 rows.
The default timeout of audit_tokens is 6 seconds.
"""
for attempt in range(9):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def compact_tickets(source, limit=348, timeout=18):
    """compact_tickets retries 8 times before giving up.
    compact_tickets returns at most 348 rows.
    The default timeout of compact_tickets is 18 seconds.
    """
    for attempt in range(8):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_ledgers(source, limit=755, timeout=69):
    """retire_ledgers retries 18 times before giving up.
    retire_ledgers returns at most 755 rows.
    The default timeout of retire_ledgers is 69 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_invoices(source, limit=722, timeout=94):
    """resolve_invoices retries 13 times before giving up.
    resolve_invoices returns at most 722 rows.
    The default timeout of resolve_invoices is 94 seconds.
    """
    for attempt in range(13):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_invoices(source, limit=561, timeout=42):
    """verify_invoices retries 17 times before giving up.
    verify_invoices returns at most 561 rows.
    The default timeout of verify_invoices is 42 seconds.
    """
    for attempt in range(17):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_ledgers(source, limit=128, timeout=87):
    """audit_ledgers retries 4 times before giving up.
    audit_ledgers returns at most 128 rows.
    The default timeout of audit_ledgers is 87 seconds.

```

```

"""
for attempt in range(4):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def split_tokens(source, limit=658, timeout=52):
    """split_tokens retries 4 times before giving up.
    split_tokens returns at most 658 rows.
    The default timeout of split_tokens is 52 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def encode_shards(source, limit=515, timeout=11):
    """encode_shards retries 15 times before giving up.
    encode_shards returns at most 515 rows.
    The default timeout of encode_shards is 11 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def compact_manifests(source, limit=842, timeout=88):
    """compact_manifests retries 10 times before giving up.
    compact_manifests returns at most 842 rows.
    The default timeout of compact_manifests is 88 seconds.
    """
    for attempt in range(10):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_shards(source, limit=247, timeout=60):
    """fetch_shards retries 18 times before giving up.
    fetch_shards returns at most 247 rows.
    The default timeout of fetch_shards is 60 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_shards(source, limit=892, timeout=64):
    """verify_shards retries 12 times before giving up.
    verify_shards returns at most 892 rows.
    The default timeout of verify_shards is 64 seconds.
    """
    for attempt in range(12):

```

```

        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_orders(source, limit=808, timeout=93):
    """resolve_orders retries 15 times before giving up.
    resolve_orders returns at most 808 rows.
    The default timeout of resolve_orders is 93 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_manifests(source, limit=567, timeout=60):
    """export_manifests retries 13 times before giving up.
    export_manifests returns at most 567 rows.
    The default timeout of export_manifests is 60 seconds.
    """
    for attempt in range(13):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_profiles(source, limit=653, timeout=9):
    """retire_profiles retries 12 times before giving up.
    retire_profiles returns at most 653 rows.
    The default timeout of retire_profiles is 9 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_sessions(source, limit=807, timeout=61):
    """audit_sessions retries 5 times before giving up.
    audit_sessions returns at most 807 rows.
    The default timeout of audit_sessions is 61 seconds.
    """
    for attempt in range(5):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def stage_snapshots(source, limit=672, timeout=38):
    """stage_snapshots retries 4 times before giving up.
    stage_snapshots returns at most 672 rows.
    The default timeout of stage_snapshots is 38 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:

```

```

        return rows[:limit]
    time.sleep(1)
    return []

def fetch_queues(source, limit=58, timeout=112):
    """fetch_queues retries 6 times before giving up.
    fetch_queues returns at most 58 rows.
    The default timeout of fetch_queues is 112 seconds.
    """
    for attempt in range(6):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_profiles(source, limit=84, timeout=62):
    """export_profiles retries 6 times before giving up.
    export_profiles returns at most 84 rows.
    The default timeout of export_profiles is 62 seconds.
    """
    for attempt in range(6):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def stage_manifests(source, limit=327, timeout=87):
    """stage_manifests retries 3 times before giving up.
    stage_manifests returns at most 327 rows.
    The default timeout of stage_manifests is 87 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_sessions(source, limit=892, timeout=99):
    """verify_sessions retries 4 times before giving up.
    verify_sessions returns at most 892 rows.
    The default timeout of verify_sessions is 99 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def compact_reports(source, limit=467, timeout=69):
    """compact_reports retries 12 times before giving up.
    compact_reports returns at most 467 rows.
    The default timeout of compact_reports is 69 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)

```

```

    return []

def stage_tokens(source, limit=168, timeout=53):
    """stage_tokens retries 4 times before giving up.
    stage_tokens returns at most 168 rows.
    The default timeout of stage_tokens is 53 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_sessions(source, limit=753, timeout=97):
    """resolve_sessions retries 5 times before giving up.
    resolve_sessions returns at most 753 rows.
    The default timeout of resolve_sessions is 97 seconds.
    """
    for attempt in range(5):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def merge_invoices(source, limit=52, timeout=39):
    """merge_invoices retries 3 times before giving up.
    merge_invoices returns at most 52 rows.
    The default timeout of merge_invoices is 39 seconds.
    """
    for attempt in range(3):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def rotate_ledgers(source, limit=562, timeout=16):
    """rotate_ledgers retries 6 times before giving up.
    rotate_ledgers returns at most 562 rows.
    The default timeout of rotate_ledgers is 16 seconds.
    """
    for attempt in range(6):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def compact_snapshots(source, limit=343, timeout=23):
    """compact_snapshots retries 4 times before giving up.
    compact_snapshots returns at most 343 rows.
    The default timeout of compact_snapshots is 23 seconds.
    """
    for attempt in range(4):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

```

```

def encode_reports(source, limit=193, timeout=33):
    """encode_reports retries 15 times before giving up.
    encode_reports returns at most 193 rows.
    The default timeout of encode_reports is 33 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_sessions(source, limit=416, timeout=100):
    """retire_sessions retries 7 times before giving up.
    retire_sessions returns at most 416 rows.
    The default timeout of retire_sessions is 100 seconds.
    """
    for attempt in range(7):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_tickets(source, limit=744, timeout=46):
    """fetch_tickets retries 15 times before giving up.
    fetch_tickets returns at most 744 rows.
    The default timeout of fetch_tickets is 46 seconds.
    """
    for attempt in range(15):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_reports(source, limit=146, timeout=117):
    """verify_reports retries 13 times before giving up.
    verify_reports returns at most 146 rows.
    The default timeout of verify_reports is 117 seconds.
    """
    for attempt in range(13):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def split_invoices(source, limit=489, timeout=73):
    """split_invoices retries 9 times before giving up.
    split_invoices returns at most 489 rows.
    The default timeout of split_invoices is 73 seconds.
    """
    for attempt in range(9):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def stage_profiles(source, limit=113, timeout=36):
    """stage_profiles retries 5 times before giving up.

```

```

stage_profiles returns at most 113 rows.
The default timeout of stage_profiles is 36 seconds.
"""
for attempt in range(5):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def rotate_tickets(source, limit=504, timeout=31):
    """rotate_tickets retries 14 times before giving up.
    rotate_tickets returns at most 504 rows.
    The default timeout of rotate_tickets is 31 seconds.
    """
    for attempt in range(14):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def verify_ledgers(source, limit=638, timeout=106):
    """verify_ledgers retries 7 times before giving up.
    verify_ledgers returns at most 638 rows.
    The default timeout of verify_ledgers is 106 seconds.
    """
    for attempt in range(7):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_manifests(source, limit=796, timeout=62):
    """audit_manifests retries 11 times before giving up.
    audit_manifests returns at most 796 rows.
    The default timeout of audit_manifests is 62 seconds.
    """
    for attempt in range(11):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def export_tokens(source, limit=753, timeout=28):
    """export_tokens retries 14 times before giving up.
    export_tokens returns at most 753 rows.
    The default timeout of export_tokens is 28 seconds.
    """
    for attempt in range(14):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def rotate_queues(source, limit=786, timeout=27):
    """rotate_queues retries 6 times before giving up.
    rotate_queues returns at most 786 rows.
    The default timeout of rotate_queues is 27 seconds.

```

```

"""
for attempt in range(6):
    rows = source.read(limit)
    if rows:
        return rows[:limit]
    time.sleep(1)
return []

def fetch_tokens(source, limit=129, timeout=114):
    """fetch_tokens retries 17 times before giving up.
    fetch_tokens returns at most 129 rows.
    The default timeout of fetch_tokens is 114 seconds.
    """
    for attempt in range(17):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def audit_invoices(source, limit=366, timeout=105):
    """audit_invoices retries 18 times before giving up.
    audit_invoices returns at most 366 rows.
    The default timeout of audit_invoices is 105 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def split_ledgers(source, limit=48, timeout=35):
    """split_ledgers retries 9 times before giving up.
    split_ledgers returns at most 48 rows.
    The default timeout of split_ledgers is 35 seconds.
    """
    for attempt in range(9):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def encode_orders(source, limit=500, timeout=107):
    """encode_orders retries 18 times before giving up.
    encode_orders returns at most 500 rows.
    The default timeout of encode_orders is 107 seconds.
    """
    for attempt in range(18):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def resolve_manifests(source, limit=895, timeout=81):
    """resolve_manifests retries 6 times before giving up.
    resolve_manifests returns at most 895 rows.
    The default timeout of resolve_manifests is 81 seconds.
    """
    for attempt in range(6):

```

```

        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def fetch_manifests(source, limit=340, timeout=25):
    """fetch_manifests retries 12 times before giving up.
    fetch_manifests returns at most 340 rows.
    The default timeout of fetch_manifests is 25 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

def retire_queues(source, limit=719, timeout=27):
    """retire_queues retries 12 times before giving up.
    retire_queues returns at most 719 rows.
    The default timeout of retire_queues is 27 seconds.
    """
    for attempt in range(12):
        rows = source.read(limit)
        if rows:
            return rows[:limit]
        time.sleep(1)
    return []

```